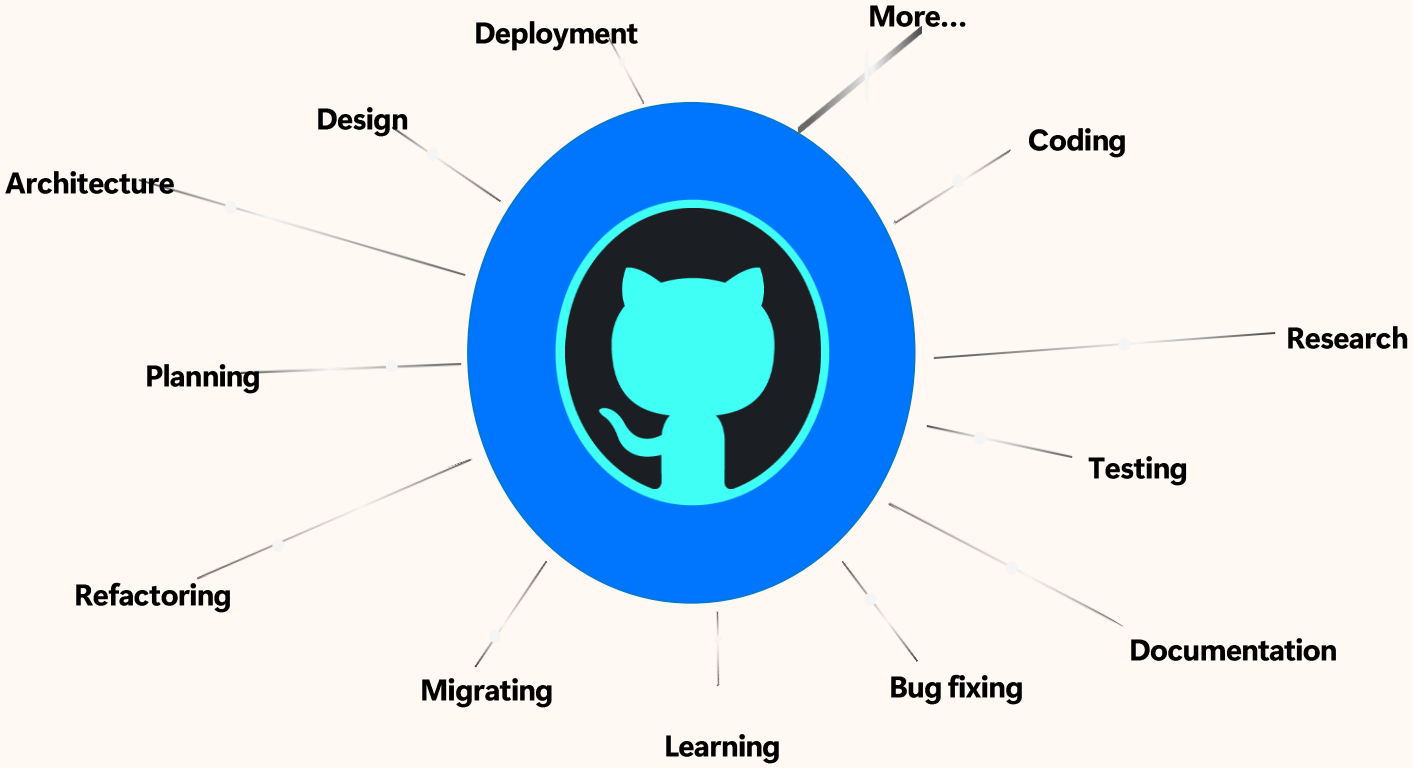
Microsoft

GitHub Copilot Jumpstart

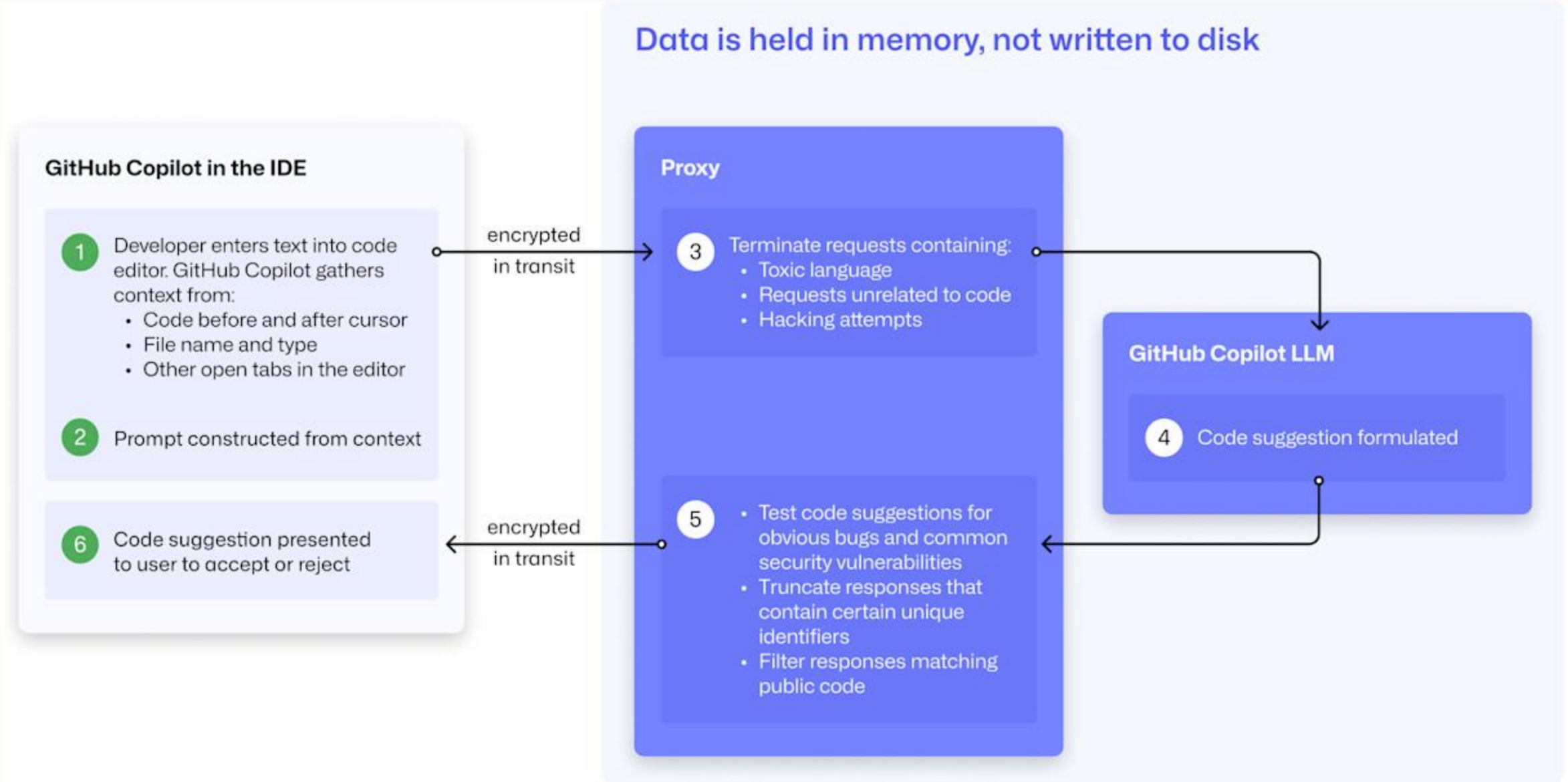
GitHub Copilot is an AI assistant used across the SDLC by various personas to turn context (code, docs, tickets, logs) into artefacts that speed up delivery and improve quality.



I'm powered by AI, so surprises and mistakes are possible. Make sure to verify any generated code or suggestions, and share feedback so that we can learn and improve

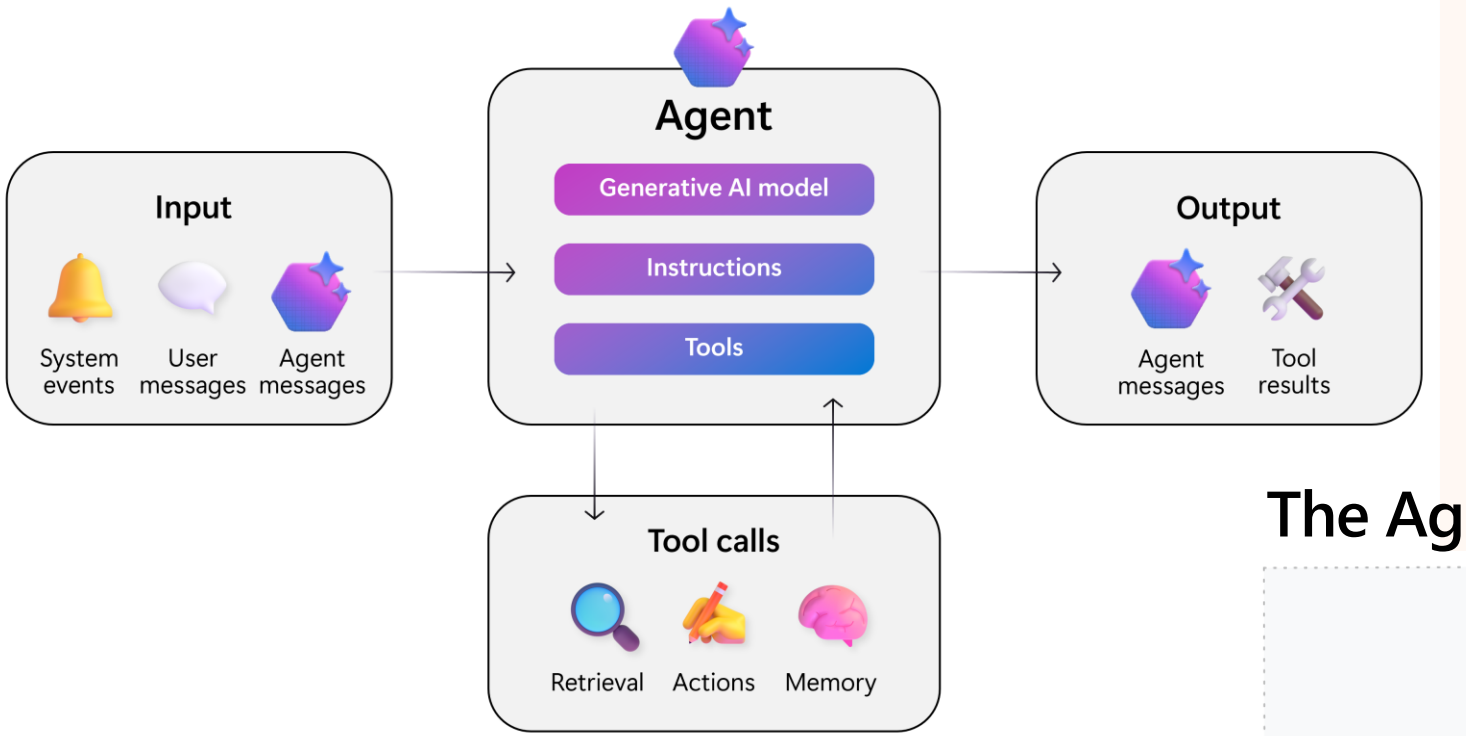
Available in various IDE's and places – with different plans

Lifecycle of code suggestion

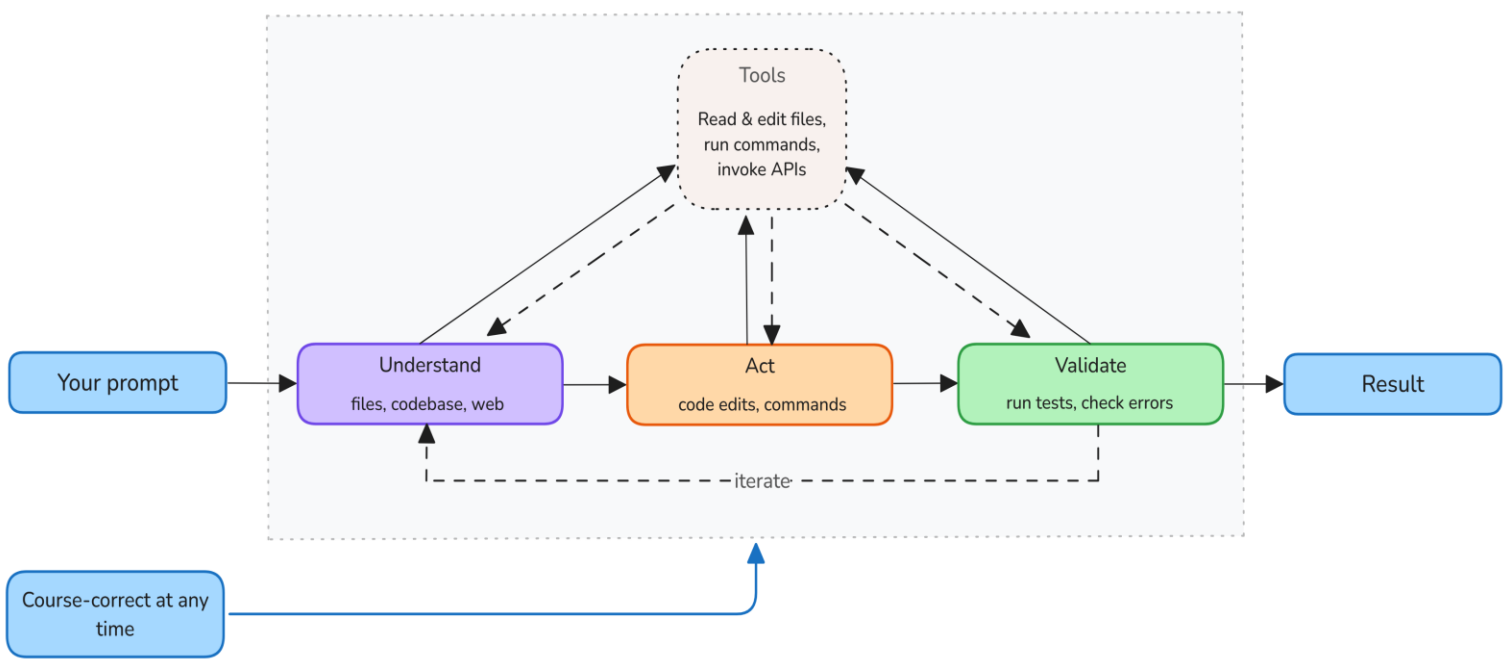


https://resources.github.com/learn/pathways/copilot/essentials/how-github-copilot-handles-data/

Agent=Models+ Tools + Instructions



The Agent Loop



Prompt Engineering

What

Prompt engineering is the process of designing and creating high-quality prompts that can be used to generate accurate and useful code suggestions with Copilot.

Why

- Maximizes utility of Copilot
- Minimizes repeated iterations
- Determines code suggestion quality
- Skill prompts guide Copilot to understand context and nuances
- Refined interactions reduces misunderstandings

How

- Zero-shot prompting
- One-shot prompting
- Few-shot prompting
- Role prompting
- Neighboring tabs
- Let's think step by step

Cornerstones

1. Context: Provide examples Information to help GitHub Copilot understand the task better
2. Clarity: Easy to understand
Being clear, transparent, and easily understood.

3. Intent: The specific goal or purpose you have in mind when creating a prompt.
4.Specificity: Providing precise and detailed information, leaving little room for ambiguity or interpretation.

Context Engineering

Providing the right information to the LLM to get a more reliable result

Practice of **designing, assembling, and maintaining the "context" you send to an LLM** (instructions + relevant information + tools + constraints)

- model reliably produces the outcome you want.

Broader than prompt engineering

- **build a context pipeline** that decides *what to include*, *in what format*, *in what order*, and *how to keep it accurate and bounded*.

Toolbox – Custom Agents, Prompts, Instructions, Skills, MCPs, hooks, plugins

Take heed of the context window

How did we do

Checklist	
Working Scenario	Maybe
Reviewed code	No
Tests	Maybe
Exception Handling	Maybe?
Monitoring/Telemetry	Maybe
Adhering to Standards/ Correct frameworks	Maybe?
Met Functional Requirements	Maybe
Secure	Maybe?
Scale and Performance..	Maybe?
Readable and maintainable	Maybe?
Documentation	Maybe?

But… Coding is now cheaper… You can implement and throw away code especially for prototype and POC..

Prompt and Context Engineering

- Prompt Engineering
  - Process of designing and creating high quality prompts
  - Used to generate useful and accurate results
  - Different techniques (zero shot, few shot, persona, think step by step etc.)

  - Context
  - Clarity
  - Intent
  - Specificity

Telling agent what to do..

- Context Engineering
  - Providing the right information to the LLM to get a more reliable result
  - Broader than prompt engineering
  - build a context pipeline that decides what to include, in what format, in what order, and how to keep it accurate and bounded.

- Toolbox – Custom Agents, Prompts, Instructions, Skills, MCPs, hooks, plugins

- Take heed of the context window

Providing agent with all relevant information

How can we improve..

Craft good prompts

- Prompt Engineering

Review the generated code. You are responsible

Understand your “environment” before jumping in – understand code and more

- Context engineering

Plan before implement. Break down into smaller tasks

- Context engineering

Guide the AI Agent – guardrails, instructions, customizations

- Context Engineering

SDLC Phases

Plan

- Structured requirements derivation from High-Level Specs
- User Story Generation

Design

- Create the design artifacts: solution design, Architecture diagrams.
- Review architecture and system diagrams for quick approval

Develop

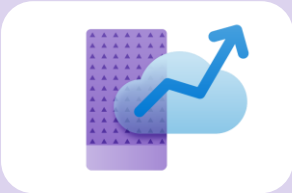
- Infra Setup Aligned to Design
- Database Scripts Ready to deploy
- Application Development from Scratch

Test

- Backlog-Driven Functional Test Design
- Functional Test Automation Script Generation

Release

- Automating Release Pipelines (CI/CD)



Maintain

- Understand unfamiliar codebases
- Fix bugs & Code Review

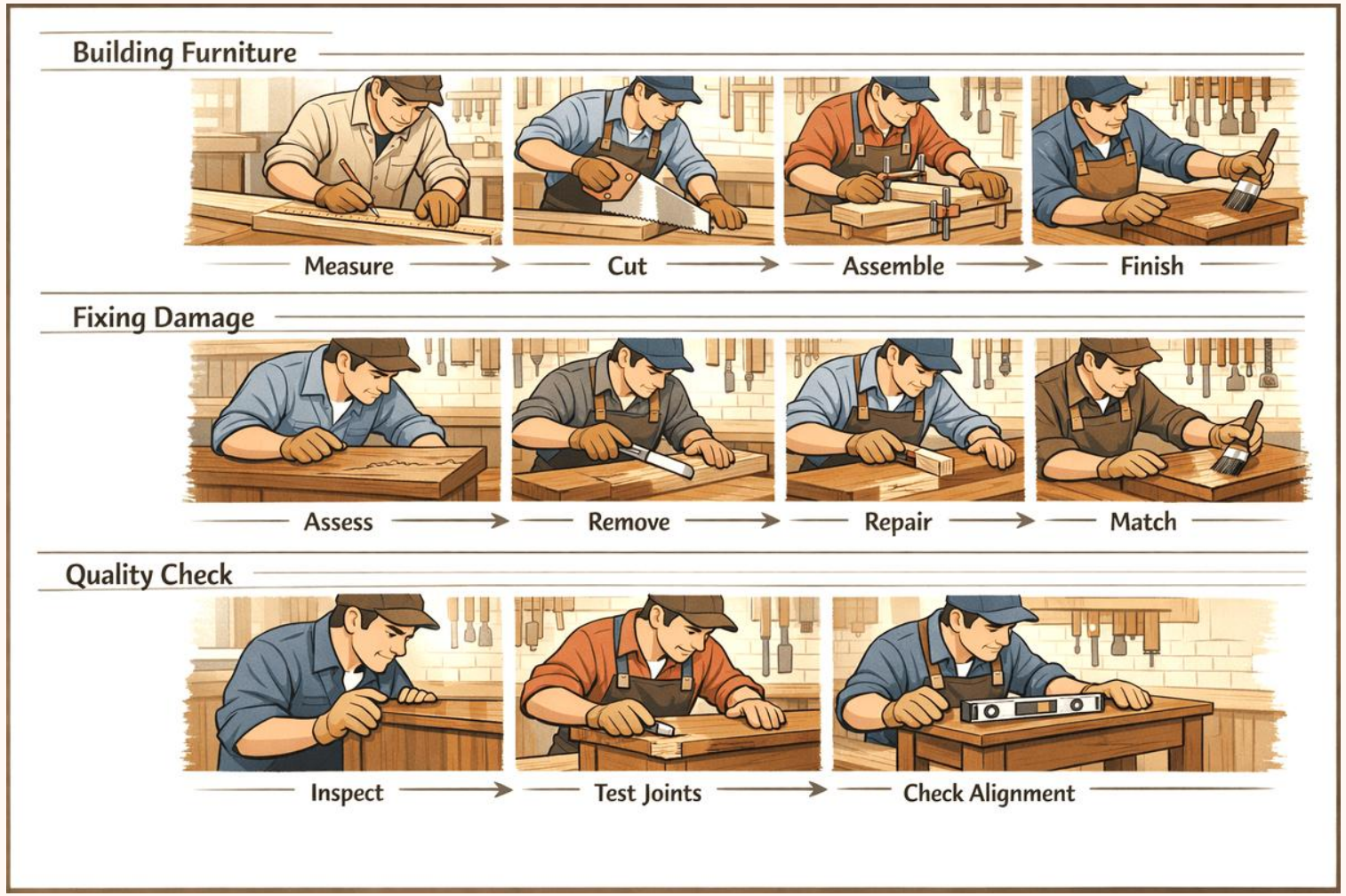
Output

- Detailed Requirement Document
- Well-Defined Backlog Item
- Solution design artifacts
- Review-ready Architecture & system diagrams
- Infrastructure set up per design.
- App built to specs from scratch to the agreed design and wireframes.
- Functional Test Cases
- Automated Test Script generation
- Script for Deployment

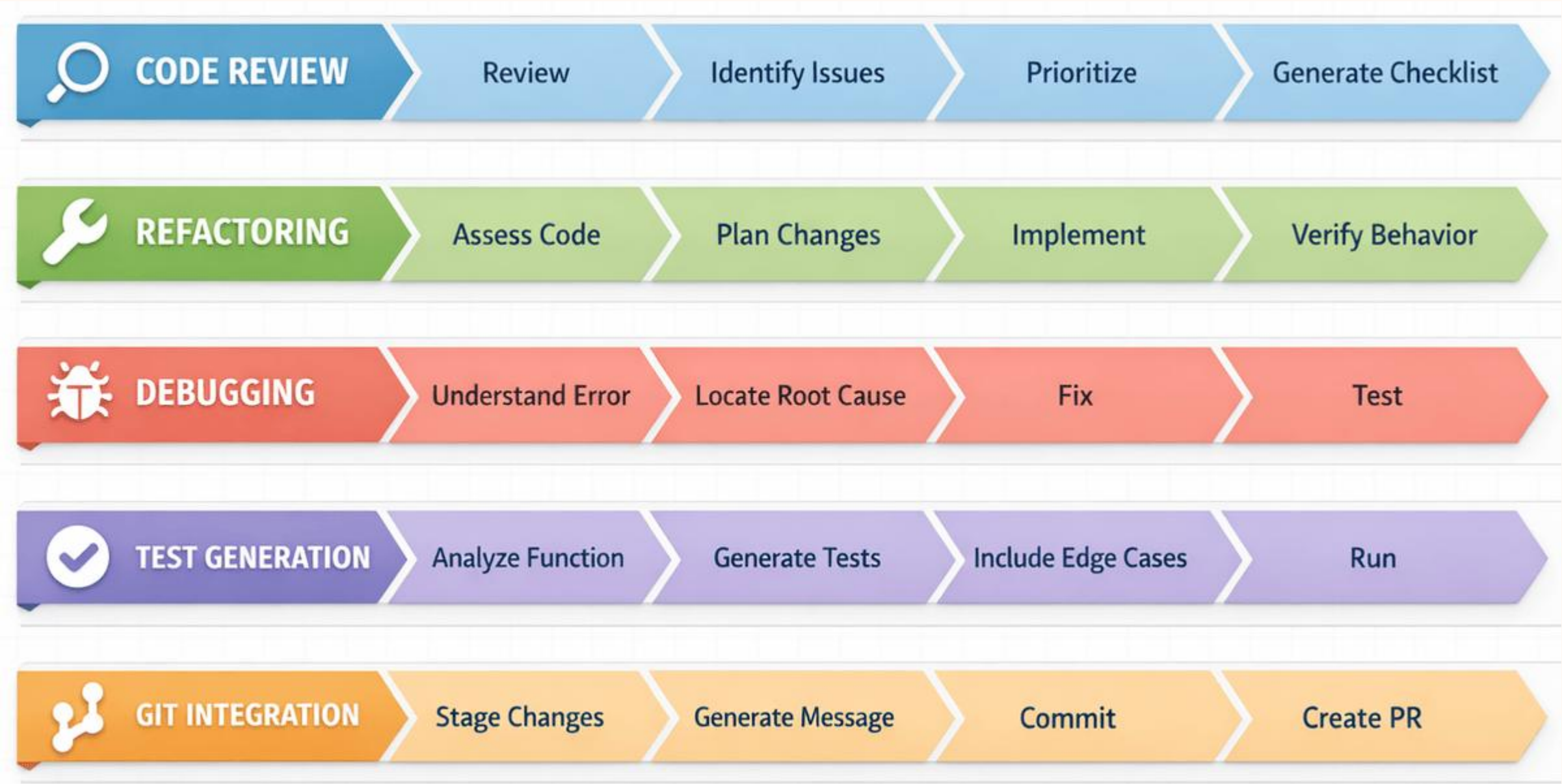
Agent Driven Development Process

- Think about your current way of working
  - Team with different roles
  - processes with a workflow
  - tools to support the processes.
- Create your own “workflows”
  - build vs “buy” vs customize
- Microsoft (open source)
  - https://github.com/microsoft/devsquad-copilot
  - https://github.com/bradygaster/squad
  - https://github.com/github/spec-kit
  - https://github.com/microsoft/hve-core
- Others -- Review them fully. Provided purely for informational purposes
  - https://github.com/obra/superpowers
  - https://github.com/steveyegge/beads
  - https://github.com/MrLesk/Backlog.md
  - ShepAlderson/copilot-orchestra
  - bigguy345/Github-Copilot-Atlas
  - https://github.com/oliguo/vibe-coding-kiro-agent
  - https://github.com/gsd-build/get-shit-done

The task determines the workflow



Different workflows based on the scenario



Customizations- the foundations of building workflows

Goal	Use	When it activates
Apply coding standards everywhere	Always-on instructions	Automatically included in every request
Different rules for different file types	File-based instructions	When files match a pattern or description
Reusable task I run repeatedly	Prompt files	When you invoke a slash command
Package multi-step workflow with scripts	Agent skills	When the task matches the skill description
Specialized AI persona with tool restrictions	Custom agents	When you select it or another agent delegates to it
Connect to external APIs or databases	MCP	When the task matches a tool description
Automate tasks at agent lifecycle points	Hooks	When the agent reaches a matching lifecycle event

Examples

Need	Use	Why
Core codebase rules	**Always-on Instructions**	Lean, every session
Rules for /tests folder	**File-based Instructions**	applyTo is deterministic
Run and fix tests quickly	**Prompt**	One-shot, user-invoked
Parse PowerPoint files	**Skill**	Capability + scripts
Constrained workflow	**Custom Agent**	Tools + system prompt enforced
Planning with review	**Top-level Agent**	Orchestrates the work
Isolated context-heavy task	**Subagent**	Returns result only
External API with auth	**MCP**	Security boundary

Practices

https://code.visualstudio.com/docs/copilot/best-practices

1. Optimize your project for AI
2. Pick the right tool for the task
3. Write effective prompts
4. Provide the right context
5. Choose the right model
6. Plan first, then implement
7. Review and verify AI output
8. Manage context and sessions

https://code.visualstudio.com/docs/copilot/guides/prompt-engineering-guide
https://code.visualstudio.com/docs/copilot/guides/context-engineering-guide

Maximize GitHub Copilot's Potential

1. Code Completion
2. Code Refactoring & Optimization
3. Documentation & Comment Generation
4. Test Generation
5. Bug Fixing & Debugging Assistance
6. Code Conversion
7. Infrastructure as Code & Automation
8. Copilot in the CLI
9. SQL Query Optimization
10. Learning & Skill Development

++ more..

Adopting Copilot

- **Will share on https://github.com/ahmedsza/ghcpcontent**
- Keep Learning, adapting – YouTube videos
- Microsoft Learn – search for “github copilot”
  - https://learn.microsoft.com/en-za/training/browse/?terms=github%20copilot
- Github Skills
  - https://learn.github.com/skills
- Culture, Culture, Culture
- Change Management
  - https://github.com/github/ai-adoption-playbook

We've learned that **AI adoption is fundamentally a people problem, not a technology problem.** Success isn't about buying the right tools—it's about building the human infrastructure that turns hesitant employees into confident power users.

Summary

1. Agent=Model + Tools + Instructions
2. Craft good prompts – Prompt Engineering
3. Context is critical -> Context Engineering
4. You are responsible for reviewing the output - always review and own the code
5. Plan and Break down the work
6. Copilot is available in many places – integrated into your existing tools and workflows – VS Code, Visual Studio, JetBrains, Eclipse, Xcode, Web, CLI, etc
7. Skills, MCP, agents, instructions and customization capabilities create possibilities
8. It is for “everyone” – not just developers
9. It is a collaboration tool that improves over time. Keep learning and adapting
10. From
    - chat → workflow
    - “AI that writes code” → “AI that understands your environment and can act safely”.
11. It is not a replacement for a human
12. You need to design your workflows

extras

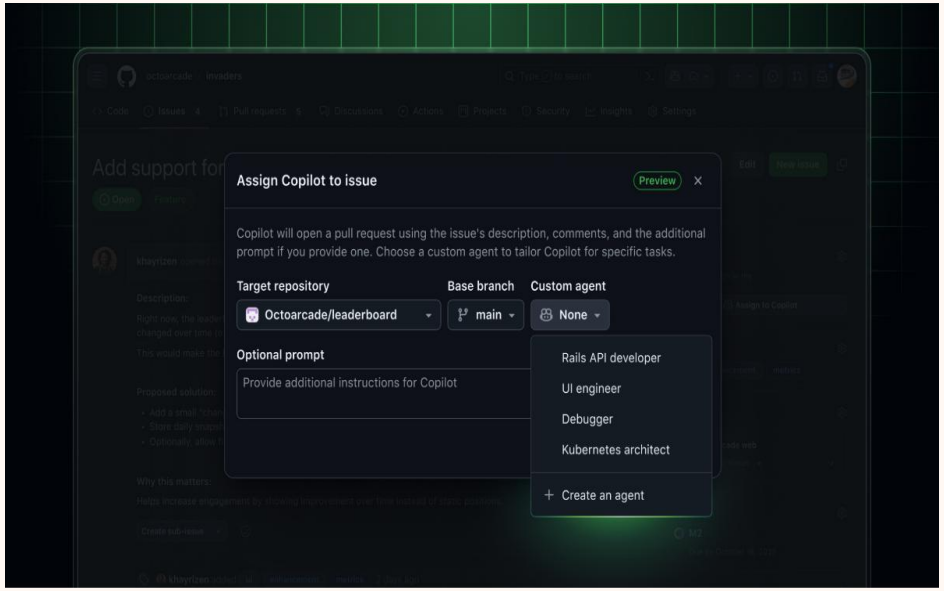
Custom Agents

Configure AI to adopt different personas for specific roles & tasks

Each persona has its own:

- Behavior & instructions
- Available tools
- Specialized focus

- Subagents
  - Independent AI agents for delegating focused subtasks (research, code analysis, reviews)
  - Own context window, can run in parallel



UI Engineer

Builds UI changes that match your component patterns.

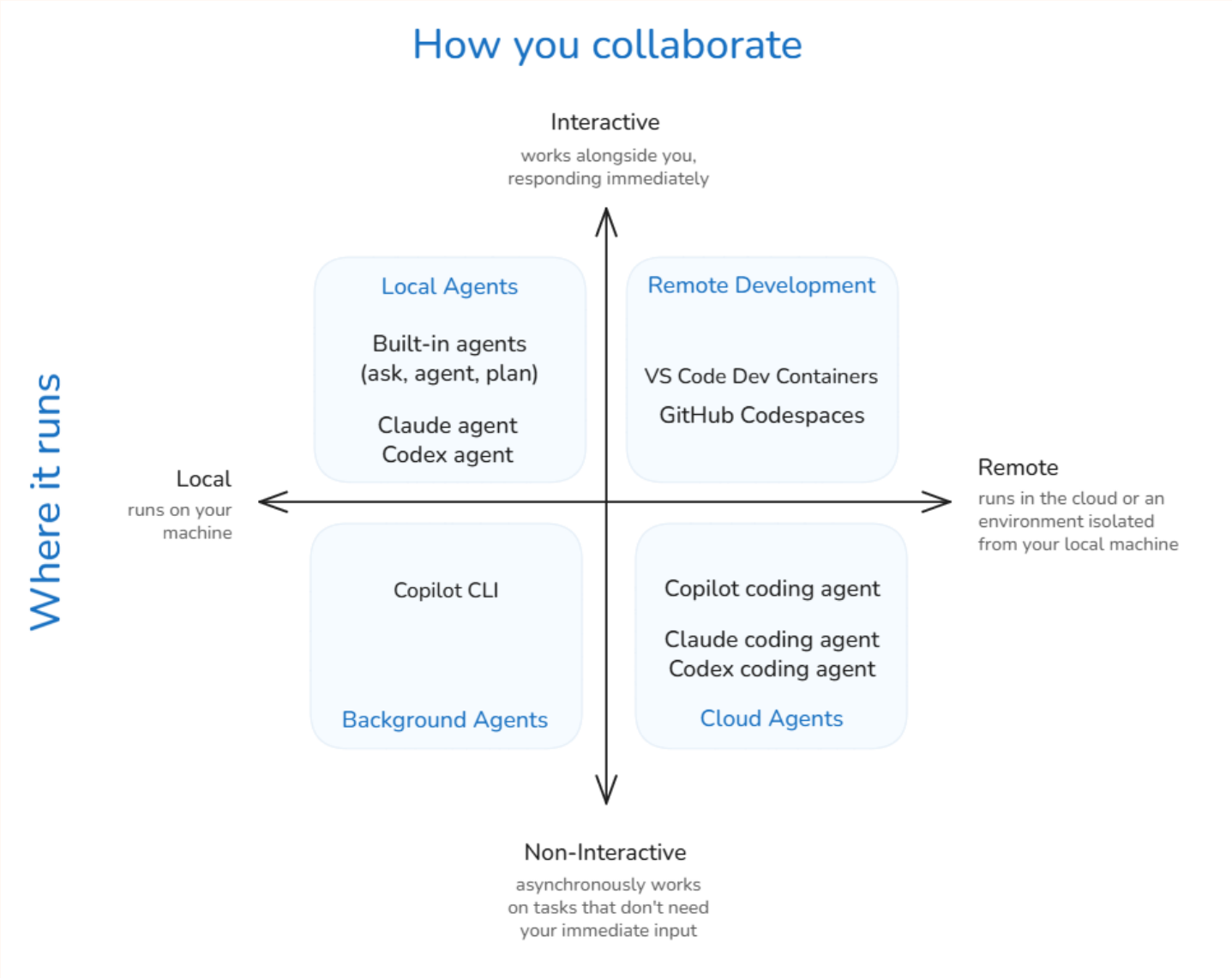
Debugger

Finds the root cause and proposes the smallest safe fix.

Kubernetes Architect

Makes deployable cluster-ready changes with sane defaults that match internal templates, tone and level of detail

Types of Agents



Creating Copilot Spaces

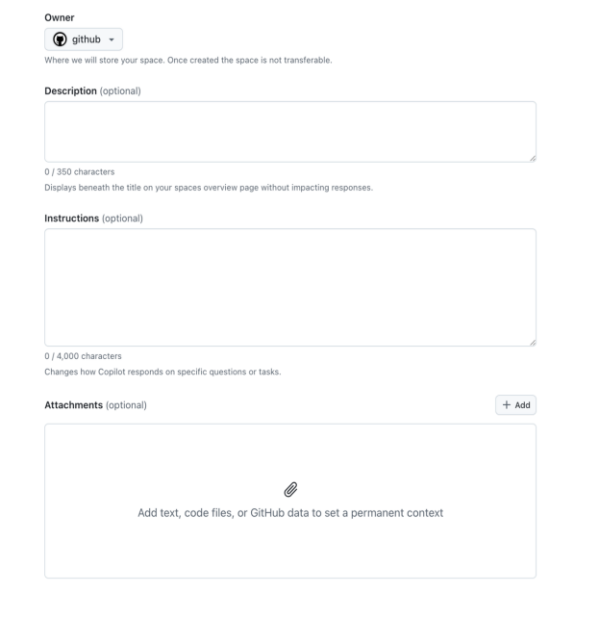
Who can create	Anyone with a Copilot license
Owned by	Organizations or individual users
Content type	Any code, pull requests, or issues hosted in repositories and free-text content
Context handling	Limited in size, which guarantees higher response quality given the focused selection

Instructions

Include areas of expertise, what kinds of tasks it should help with, and what it should avoid to keep responses relevant

Adding context

Add text, code files, or GitHub data to set a permanent context. This will be used to provide more relevant answers.



Code Completions

Code completions

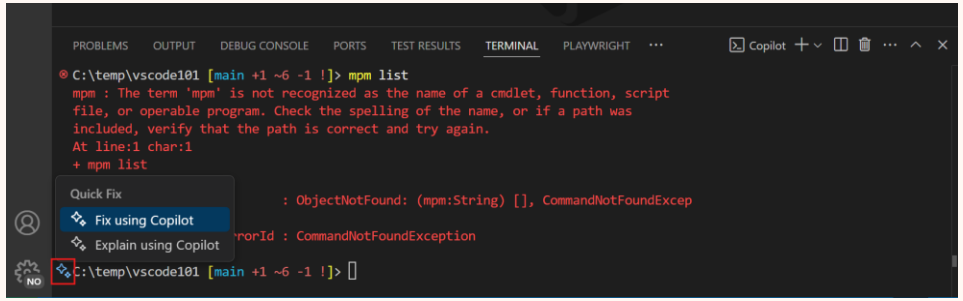
- Start typing in the editor, and Copilot provides code suggestions
- match your coding style and take your existing code into account.
- Can show alternative suggestions and accept partial suggestions
- Can show in a new tab
- Can be disabled.

Next edit suggestions

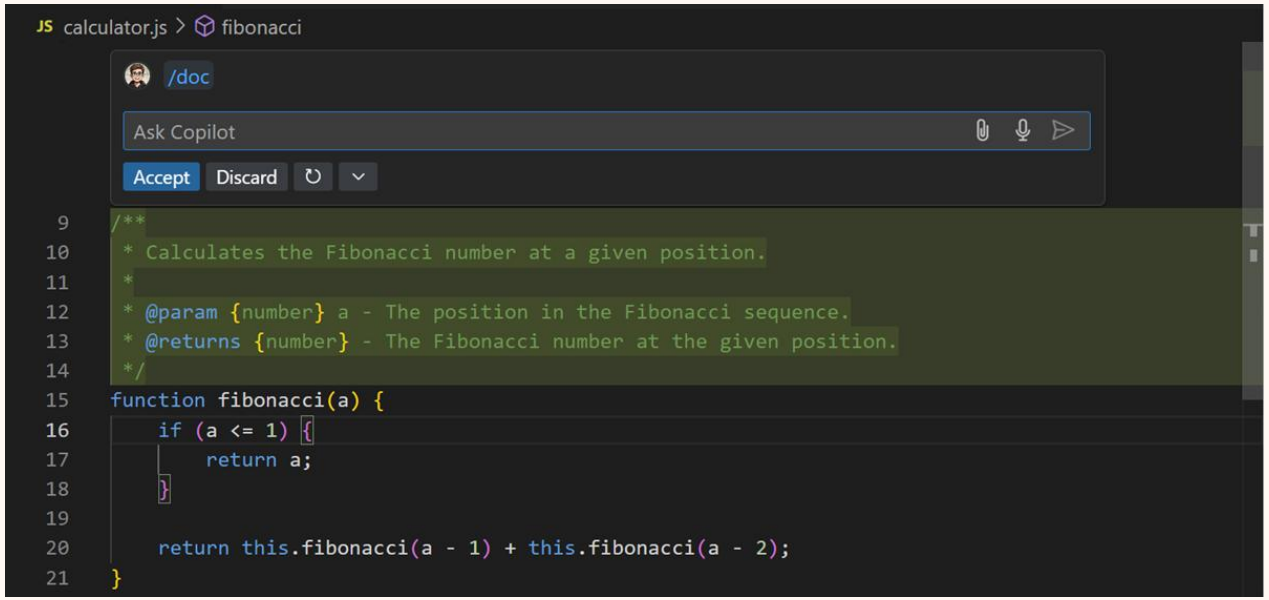
- Predict your next code edit with Copilot next edit suggestions, aka Copilot NES.
- Based on the edits you're making, NES both predicts the location of the next edit you'll want to make and what that edit should be.
- Use cases - simple mistakes like typos, challenging mistakes in logic, Changing intent, Refactoring

Smart Actions

- For several common scenarios, you can use *smart actions* to get help from AI without having to write a prompt.
  - Examples of these smart actions are generating commit messages, generating documentation, explaining or fixing code, or performing a code review.



- Scenarios
  - *Generate a commit message and PR information*
  - *Implement todo comments*
  - *Rename symbols*
  - *Generate alt text for images in Markdown*
  - *Generate documentation*
  - *Generate tests*
  - *Explain code*
  - *Fix coding errors*
  - *Fix testing errors*
  - *Fix terminal errors*
  - *Review code*
  - *Semantic search results (Preview)*
  - *Search settings with AI*



GitHub Copilot app modernization

Modernize your apps in days, not months!

Accelerate Java and .NET upgrades
Upgrade language runtime versions and frameworks effortlessly with AI-driven remediation and automated error fixes—no more dependency headaches.

Streamline end-to-end migration
From assessment to deployment, simplify the migration journey with full visibility, control, and predictability, while delivering higher quality outcomes.

Become Cloud and AI ready
Make apps cloud-ready by resolving compatibility issues, reducing technical debt, and strengthening security while unlocking the full power of Azure services.

Included in GitHub Copilot License

Available in VS Code, Visual Studio

Key Capabilities	Description
Assessment & Planning	Code assessment made easy. GitHub Copilot reviews your .NET and Java codebase to generate modernization plans, helping you prioritize efforts.
Code Transformations	Automated transformation. Copilot combines expert-led Azure migration strategies with automated upgrades, applying high-confidence changes with precision.
Patching CVE and security checks	Security is built-in. GitHub Copilot scans for known CVEs and flags outdated dependencies. In agent mode, it can auto-fix issues—letting you review and approve changes.
Build, Validate and Tests	Test coverage enhanced. Existing test cases are migrated, and new ones are generated automatically—ensuring consistency and compliance.
Containerization	Effortless containerization. Copilot auto-generates Docker files to help you containerize your app with minimal effort.
Deployment	Deployment-ready artifacts. Copilot generates deployment files and supporting assets to accelerate dev/test setup and build confidence for production.

Get Started

Get started with modernizing your Java and .NET apps today and learn more about the technical details here.

Watch the end-to-end introduction videos for Java and .NET.

Get the guidance deck and field FAQ on Seismic

Spec Kit https://github.com/github/spec-kit

Spec Driven Development

- flips the script on traditional software development.
- specifications become executable
- directly generating working implementations rather than just guiding them.
- Intent-driven development where specifications define the "what" before the "how"
- Rich specification creation using guardrails and organizational principles
- Multi-step refinement rather than one-shot code generation from prompts
- Heavy reliance on advanced AI model capabilities for specification interpretation

Command	Description
`/constitution`	Create or update project governing principles and development guidelines
`/specify`	Define what you want to build (requirements and user stories)
`/clarify`	Clarify underspecified areas (must be run before `/plan` unless explicitly skipped; formerly `/quizme`)
`/plan`	Create technical implementation plans with your chosen tech stack
`/tasks`	Generate actionable task lists for implementation
`/analyze`	Cross-artifact consistency & coverage analysis (run after /tasks, before /implement)
`/implement`	Execute all tasks to build the feature according to the plan